E-commerce Customer Analytics: End-to-End Project Documentation

Domain: Indian e-commerce (retail across Electronics, Furniture, Home & Kitchen, Office Supplies, Fashion) **Data period:** 1 Jan 2021 – 31 Dec 2024 **Stack:** PostgreSQL, Python (pandas, NumPy, SQLAlchemy, psycopg2), SQL, Jupyter, Power BI, Excel/CSV

---

1. Executive Summary

This project builds a complete analytics pipeline for an e-commerce business: raw order, customer and product data is loaded into PostgreSQL, extracted and cleaned with Python, enriched with **RFM customer segmentation** and **churn classification**, written back to the database, analysed with SQL business queries, and presented in a Power BI dashboard.

The headline findings:

- The business generated about **₹44.07 Cr in sales and ₹4.27 Cr in profit** (9.7% margin) across 11,000 orders and 720 customers.
- **Furniture is 33% of revenue but earns only a 1.5% margin**, and the single product *Pepperfry Sofa Set* (₹5.67 Cr sales) is loss-making overall.
- **Discounting is the biggest profit leak**: margin falls from 12.1% on undiscounted lines to 2.6% on lines discounted 20–30%.
- **About 33% of lifetime customer revenue (₹14.38 Cr) sits with 252 customers who have gone quiet** (Churn Risk or Likely Churned).

---

2. Business Problem

The business had transactional data but no single view that answered the questions management actually cares about:

1. How are sales and profit trending, and where do they come from (region, state, category, product, payment mode)?
2. Which products and discount levels are quietly destroying profit?
3. Who are our best customers, and how do we recognise them?
4. Which customers are drifting away, and how much revenue is tied to them?

5. Can the above be refreshed automatically rather than rebuilt by hand each time?

In short: **raw data was spread across three tables, had quality risks, and gave no customer-level insight or early warning on churn.**

---

3. Objectives

- Design a normalised relational database for customers, products and orders.
- Build a repeatable ETL pipeline that cleans data and flags quality issues.
- Segment customers using RFM and classify churn status with revenue at risk.
- Answer core business questions in SQL.
- Deliver an interactive Power BI dashboard for decision makers.
- Turn insights into concrete, prioritised recommendations.

---

4. Data Description

Table / file	Rows	Key columns
`customers.csv`	720	customer_id, customer_name, segment, city, state, region
`products.csv`	28	product_id, category, sub_category, product_name, min_price, max_price, base_margin
`orders.csv`	17,107 order lines (11,000 unique orders)	row_id, order_id, order_date, ship_date, ship_mode, customer_id, product_id, quantity, discount, sales, profit, payment_mode

Outputs produced by the pipeline: `clean_orders` (27 columns), `customer_rfm` (720 rows), `customer_churn` (720 rows). Each is stored in PostgreSQL and exported to CSV for Power BI.

---

5. Tools Used and How Each Helped

Tool	Role in the project	How it helped solve the problem
PostgreSQL	Central data store	Enforced primary and foreign keys between customers, products and orders; made large aggregations fast and repeatable
SQL (`creation.sql`, `clean_tables.sql`, `business_queries_project.sql`)	Schema design, integrity checks, business analysis	Validated that every order maps to a real customer and product; produced the KPI, regional, category, product, RFM and churn summaries
Python / pandas / NumPy	Extraction, cleaning, feature engineering	Automated cleaning, derived new fields (profit margin, ship days, order month) and computed RFM and churn logic
SQLAlchemy + psycopg2	Database connectivity (`config.py`)	Auto-discovered the `ecommerce*` database and gave pandas a clean read/write connection

Tool	Role in the project	How it helped solve the problem
Jupyter Notebooks	Exploratory analysis (`EDA_before_cleaning`, `eda_after_cleaning`)	Profiled data before and after cleaning to prove the cleaning improved quality
Power BI (`ecommerce_analytics.pbix`)	Visualisation	Turned the cleaned tables into an interactive dashboard for non-technical stakeholders
CSV / Excel exports	Hand-off layer	Fed Power BI and allowed quick inspection of each stage's output

---

6. Architecture and Workflow

```
customers.csv / products.csv / orders.csv
            |  (loaded into PostgreSQL – creation.sql)
            ▼
   PostgreSQL: customers, products, orders (with PK/FK)
            |  extract.py  (JOIN orders + customers + products)
            ▼
   clean.py  → clean_orders      → load.py → PostgreSQL + clean_orders.csv
            ▼
   rfm.py    → customer_rfm      → load.py → PostgreSQL + rfm.csv
            ▼
   churn.py  → customer_churn    → load.py → PostgreSQL + churn.csv
            ▼
   SQL business queries  +  Power BI dashboard  →  insights and
recommendations
```

`pipeline.py` orchestrates all steps in order, so the whole workflow runs with one command.

---

7. What Was Done, Step by Step

Step 1: Database design and loading (`creation.sql`)

- Created `customers`, `products` and `orders` with primary keys and foreign keys (`orders.customer_id → customers`, `orders.product_id → products`).
- Ran integrity checks: total rows, distinct orders/products/customers, and anti-joins to confirm no order references a missing customer or product.

Step 2: Extraction (`extract.py`)

- Joined the three tables into one analysis-ready dataset (order line + customer attributes + product attributes).

Step 3: Data cleaning (`clean.py`)

- Removed duplicate rows.
- Converted `order_date` and `ship_date` to datetimes (invalid values become null and are flagged).
- Coerced numeric columns (quantity, discount, sales, profit, min/max price, base margin).
- Imputed missing values: mean for quantity, discount and base margin; **category-wise median** for sales and profit; `"Unknown"` for missing text fields.
- Created **data-quality flags**: `invalid_order_date`, `invalid_ship_date`, `missing_customer_details`, `missing_product_details`, `invalid_discount`, `negative_ship_days`.
- Engineered features: `profit_margin`, `order_year`, `order_month`, `ship_days`.
- Removed rows with negative sales or quantity and printed a data-quality summary.

Result: 17,107 clean order lines; no negative shipping days, and discounts all within the valid 0–30% range.

Step 4: RFM segmentation (`rfm.py`)

- Set the snapshot date to one day after the last order (1 Jan 2025).
- Aggregated per customer: **Recency** (days since last order), **Frequency** (distinct orders), **Monetary** (total sales), plus profit.

- Scored each metric 1–5 using rank-based quintiles (`qcut`), then built a combined `rfm_score`.
- Assigned business segments:

Segment	Rule
VIP Customers	R, F and M all ⩾ 4
Loyal Customers	R ⩾ 3 and F ⩾ 3
At-Risk Customers	R ⩽ 2 and F ⩾ 3
Lost Customers	R ⩽ 2 and F ⩽ 2
Regular Customers	Everyone else

Step 5: Churn classification (`churn.py`)

Days since last order	Status
⩽ 30	Active
31–90	Warm
91–180	Churn Risk
> 180	Likely Churned

`revenue_at_risk` equals the customer's total spend for Churn Risk and Likely Churned customers, and 0 otherwise.

Step 6: Loading back (`load.py`, `pipeline.py`)

- Wrote `clean_orders`, `customer_rfm` and `customer_churn` to PostgreSQL and exported matching CSVs.

Step 7: Business analysis (`business_queries_project.sql`)

Twelve queries covering: KPI summary, monthly trend, region, state, category, top customers, top products by profit, loss-making products, RFM summary, churn summary, top at-risk customers and payment-mode performance.

Step 8: Dashboard (Power BI)

- Connected the cleaned tables in `ecommerce_analytics.pbix` to present KPIs, trends, geography, category/product performance, RFM segments and churn exposure.

---

8. Key Findings

8.1 Overall performance

Metric	Value
Total sales	₹44.07 Cr (₹440.7 M)
Total profit	₹4.27 Cr (₹42.7 M)
Profit margin	9.7%
Orders / customers	11,000 / 720

Yearly sales were flat: ₹10.67 Cr (2021), ₹10.80 Cr (2022), ₹11.71 Cr (2023), ₹10.89 Cr (2024). Growth peaked in 2023 and fell back in 2024. Margin stayed steady at 9.6–9.8% every year, which means the problem is structural (mix and discounting), not a one-off.

8.2 Category and product performance

Category	Sales (₹ Cr)	Margin
Electronics	24.22	13.4%
Furniture	14.58	**1.5%**
Home & Kitchen	2.27	15.5%
Office Supplies	1.85	12.7%
Fashion	1.16	18.9%

- Electronics produces **76% of total profit** (₹3.25 Cr of ₹4.27 Cr). HP Pavilion and Lenovo IdeaPad laptops alone contribute about ₹2.06 Cr profit.

- Furniture earns ₹1.46 Cr sales for every ₹0.02 Cr of profit. **Sofas have a −0.2% margin** and the Pepperfry Sofa Set is the only loss-making product (about −₹12 lakh on ₹5.67 Cr sales). Storage (0.9%), Tables (1.7%), Chairs (3.3%) and Beds (3.5%) are barely profitable.
- Fashion and Home & Kitchen have the best margins (18.9% and 15.5%) but are tiny (about 3% and 5% of sales), a growth opportunity.

8.3 Discount impact

Discount band	Sales (₹ Cr)	Margin
0%	18.44	12.1%
1–10%	15.46	9.6%
11–20%	7.62	6.4%
21–30%	2.55	**2.6%**

Every step up in discount costs margin. Lines discounted over 20% generate almost no profit.

8.4 Geography and customers

- West (₹12.10 Cr) and South (₹11.66 Cr) lead; North ₹10.45 Cr, East ₹8.54 Cr; **Central is small (₹1.32 Cr, 3%) with the weakest margin (8.9%)**.
- Maharashtra is the largest state at ₹5.99 Cr (about 14% of sales), followed by Karnataka (₹2.99 Cr).
- The customer base is not concentrated: the top 10% of customers account for only about 21% of sales, so revenue risk is spread broadly rather than hanging on a few accounts.
- Consumer segment is 48% of sales; Corporate 31%.
- Payment modes: UPI leads at 45% of sales; Net Banking has the lowest margin (9.0%). Shipping mode has little effect on margin.

8.5 RFM segments

Segment	Customers	Revenue (₹ Cr)	Avg recency (days)
Loyal	192	12.45	37
At-Risk	157	10.81	152
Regular	157	7.31	39
VIP	83	7.94	21
Lost	131	5.56	193

At-Risk customers already have high purchase frequency but have not bought in about five months. Together, At-Risk and Lost customers are 288 of 720 customers (40%).

8.6 Churn exposure

Status	Customers	Lifetime revenue (₹ Cr)
Active	207	13.74
Warm	261	15.96
Churn Risk	161	9.95
Likely Churned	91	4.43

Revenue at risk: ₹14.38 Cr, about 33% of lifetime revenue. The largest single exposures are frequent buyers (13–19 orders each, spending ₹14–22 lakh) who have been inactive for 3–4 months, which makes them still very recoverable.

> Note: revenue at risk is each customer's *historical* spend, a measure of value exposed, not a forecast of future loss.

---

9. Problems Solved and Impact

Problem	What the project did	Impact
Data spread over three tables with quality risks	Relational schema, integrity checks, automated cleaning with quality flags	One trusted, analysis-ready dataset and visible data-quality metrics
Manual, non-repeatable analysis	Modular ETL pipeline run from `pipeline.py`	Refreshing all outputs is a single command
No view of customer value	RFM scoring and five segments	Marketing can target VIP, Loyal, At-Risk and Lost groups differently
No early warning on churn	Recency-based churn status and revenue at risk	252 customers and ₹14.38 Cr of exposed revenue identified, with a ranked top-20 call list
Hidden profit leaks	Category, product and discount analysis	Pinpointed Furniture/Sofas and deep discounts (>20%) as the main margin drains
Insights locked in SQL and CSVs	Power BI dashboard	Self-service reporting for non-technical stakeholders

---

10. Recommendations

A. Protect and recover profit (highest priority)

1. **Cap discounts at about 15–20%.** Lines above 20% earn a 2.6% margin; approval should be required beyond the cap, and discount use should be tracked per product.
2. **Fix Furniture pricing.** Re-negotiate supplier cost or raise prices on Sofas first (−0.2% margin on ₹5.67 Cr of sales), then Storage, Tables, Chairs and Beds. Consider cutting or bundling the Pepperfry Sofa Set if the margin cannot be restored.
3. **Shift marketing spend** toward Fashion, Home & Kitchen and Electronics accessories, which have 13–19% margins, and cross-sell them to Electronics buyers.

B. Win back at-risk customers

4. **Run a win-back campaign** for the 161 Churn Risk customers first (91–180 days): personalised offers on categories they bought, using the top-20 at-risk list as a call/outreach list for account managers.
5. **Use cheap, low-discount incentives** (free shipping, loyalty points, early access) instead of deep price cuts, to avoid trading back the margin gained in step 1.
6. **Treat Likely Churned (91) as a low-cost reactivation test** (email or WhatsApp only) rather than a heavy-spend effort.
7. **Trigger alerts at 60 days of inactivity** (before Warm turns into Churn Risk) for any customer with frequency above the median.

C. Grow the best customers

8. **Launch a VIP / Loyal programme** (tiers, rewards, priority support) for the 275 VIP and Loyal customers who generate ₹20.4 Cr, 46% of revenue.
9. **Upgrade Regular customers** (157) with frequency-building offers, since they are recent (39 days) but not yet loyal.

D. Regional and operational

10. **Investigate Central region** (3% of sales, 8.9% margin): decide between targeted expansion and de-prioritising it.
11. **Review Net Banking** (lowest margin at 9.0%) and nudge customers to UPI, which has both the highest volume and better margin.
12. **Replicate the Maharashtra and Karnataka playbook** (about 20% of sales combined) in comparable states.

---

11. Limitations and Technical Improvements

These are worth fixing before the project is shared or deployed.

Area	Observation	Suggested fix
Security	`config.py` contains the database password in plain text	Move credentials to environment variables or a `.env` file; rotate the password, and add the file to `.gitignore` before pushing to GitHub
Schema vs pipeline	`load.py` uses `if_exists="replace"`, which drops the tables created in `clean_tables.sql` and recreates them without primary/foreign keys	Use `if_exists="append"` after truncating, or re-apply constraints after loading
Flag columns	The data-quality flags created in `clean.py` are not in the `clean_orders` table definition	Add them to the SQL schema, or document that `replace` creates them
Imputation	Mean imputation can create fractional quantities and ignores product-level patterns	Use median or product-wise imputation; round quantity to whole numbers
Quality summary order	The summary prints after bad rows are filtered, so counts can under-report	Print the summary before filtering
Inner join in extraction	Orders without a matching customer or product are dropped silently, so the "Unknown" fills are rarely used	Log the number of dropped rows, or use a left join if you want to keep them
Churn thresholds	30/90/180-day cut-offs are fixed and not tied to each customer's normal buying rhythm	Compare recency to the customer's own average gap between orders
RFM scoring	Rank-based quintiles force equal-sized groups even when values are tied	Document this choice, or validate against value-based thresholds
Analysis depth	Analysis is descriptive	Add a predictive churn model (logistic regression or gradient boosting), Customer Lifetime Value, and seasonality analysis

Area	Observation	Suggested fix
Automation	Pipeline runs manually	Schedule with cron, Windows Task Scheduler or Airflow; add logging and tests

---

12. How to Run the Project

1. Create the PostgreSQL database (name starting with `ecommerce`) and run `creation.sql`, then load the three CSVs into `customers`, `products` and `orders`.
2. Run `clean_tables.sql` to create the output tables.
3. Install dependencies: `pip install pandas numpy sqlalchemy psycopg2-binary`.
4. Run `python pipeline.py`. It extracts, cleans, scores RFM, classifies churn, loads the three tables, and writes `clean_orders.csv`, `rfm.csv` and `churn.csv`.
5. Run the queries in `business_queries_project.sql` to reproduce the analysis.
6. Open `ecommerce_analytics.pbix` in Power BI and refresh the data source.

File reference

File	Purpose
`creation.sql`	Source tables and integrity checks
`clean_tables.sql`	Output tables (clean orders, RFM, churn)
`business_queries_project.sql`	Business analysis queries
`config.py`	Database connection
`extract.py`	Join and extract data
`clean.py`	Cleaning and feature engineering
`rfm.py`	RFM scoring and segmentation
`churn.py`	Churn status and revenue at risk
`load.py`	Write results to PostgreSQL
`pipeline.py`	Runs the full workflow

File	Purpose
`EDA_before_cleaning` / `eda_after_cleaning` notebooks	Exploratory analysis before and after cleaning
`ecommerce_analytics.pbix`	Power BI dashboard

---

13. Conclusion

The project turned three raw tables into a reusable analytics system that explains **where the money is made and lost** (Electronics earns, Furniture and heavy discounting drain), **who the valuable customers are** (RFM), and **who is about to leave** (churn). The most valuable next actions are capping discounts, repairing Furniture margins, and running a targeted win-back campaign on the 252 at-risk customers carrying ₹14.38 Cr of lifetime revenue.

Figures in this document were calculated from the project's `clean_orders`, `rfm` and `churn` outputs. The Power BI file and notebooks were not opened, so visuals and notebook findings are described by their purpose only.